



交互式机器人官网 小白指南

从 Microduck Anatomy 学会：把机器人"解剖"在浏览器里 —— GLB 模型 × 关节树 × 策略步态 × 逆运动学 × 爆炸图

拾遗队列 cb09e4bb · 上游: huggingface.co/spaces/mishig/microduck-anatomy (源码公开)

一句话总览：Hugging Face 工程师给自家机器鸭 Microduck 做了一个"全息解剖"官网——官网页面上那只鸭子不是视频、不是动图，而是**官方 3D 模型 + 官方强化学习步态数据在浏览器里实时跑出来的"数字展台"**：能旋转、能走路、能拖它的腿（逆运动学）、能把 70 个零件炸开分层看。本指南拆解它的五层方法论，并带你用 200 行代码复刻一个能跑能炸开的迷你版。

一、疑问即目录 (12 问)

Q1 这是什么？一个"会解剖自己"的机器人官网

传统机器人官网 = 精修图片 + 参数表格 + 视频宣传片，**用户只能"看"机器人**。Microduck Anatomy 换了个思路：把官网本身变成一台**虚拟展台**——产品自己把自己拆开给你看。它是 Hugging Face（已收购 Pollen Robotics）员工 mishig 做的开源项目，部署在 HF Space 上，源码持续更新。地址：mishig-microduck-anatomy.hf.space。

传统官网：图片 + 表格 + 视频
用户是观众 🙈



交互式官网：实时 3D 数字展台
用户是解剖师 🦷 转动/拆解/拖拽

Q2 官网的 3D 模型从哪来? ——glTF/GLB 资产管线

GLB 是三维界的"JPEG": 把网格(形状)、贴图、材质打包成一个自包含二进制文件(glTF 格式的二进制版), 浏览器里 `GLTFLoader` 一行加载。Microduck 官网的妙处: **不重新建模**, 直接用官方仿真仓库(mjlab) 里的 `microduck.glb` (1.28 MB) ——和训练机器人用的是同一份模型。

文件	角色	大小
<code>microduck.glb</code>	零件库 : 只取每个 mesh 的几何体 (geometryByName 索引)	1.28 MB
<code>kinematics.json</code>	装配图 : MuJoCo MJCF 关节树导出 (body 层级/位置/四元数/铰链轴)	28 KB
<code>gait 数据</code>	动作库 : RL 策略 50Hz 采样 32 帧 × 18 通道	6.8 KB

方法论金句: 几何体和装配关系分开存, 网页端自己"总装"。

Q3 光有模型不会动: 关节树怎么搬进浏览器?

`kinematics.json` 里有 27 个 body, 每个带 `pos` (位置)、`quat` (朝向四元数, MJCF 是 w,x,y,z 顺序, three.js 是 x,y,z,w——**要换位**)、`joint` (铰链轴 + 角度范围)。网页代码为每个 body 建一个 `THREE.Group`, 按 parent 字段拼成树, 铰链记成 JointRig。给某关节设角度 = 绕轴四元数 × 基础姿态——这正是知识库[[前向运动学FK与四元数链]]的网页实战版: 父关节转, 全身子件跟着动。



Q4 走路的数据是哪来的？——策略步态驱动网页

这是最“卷”的一步：官网的走路**不是动画师 K 的帧**，而是 Microduck 官方强化学习策略

`BEST_alpha_walking.onnx`（在 MuJoCo 里训练，见[[Sim2Real与MuJoCo仿真]]）以 50 Hz 跑出来的真实关节轨迹，采样成 **32 帧 × 18 通道**（14 个关节角 + 躯干侧偏/高度/滚转/俯仰 4 通道），周期 0.812 秒。网页端每帧按相位插值取一帧，灌进关节树。**用户在官网看到的步态，和真机器人走路的步态是同一份数据——“数字展台”的含金量就在这。**



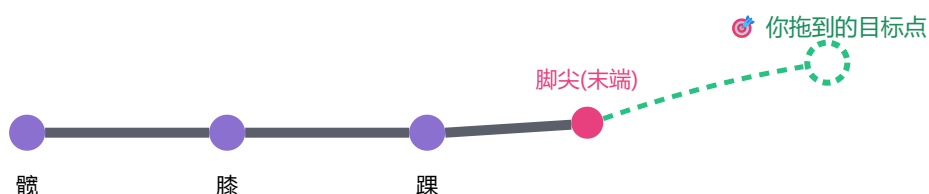
真机的“脑子”直接变成网页的“动作库”，一模一样的走路姿势

Q5 15 个舵机怎么“各就各位”？——名字分类 + 图层聚焦

网站按 mesh 名字给 70 个零件自动分类：`x1330*` → 舵机、`lens/np_f/speaker` → 传感、`pcb/robot_hat` → 计算、`shell/jaw/foot` → 外壳……点顶部图层按钮（ENTIRE RIG / SMART SERVOS / VISION / COMPUTE / POWER），不在当前层的零件透明度压到 0.02~0.07，“**分层解剖**”的本质就**是一张透明度权重表**。同时每层配一个“聚焦舞台”：镜头目标点、机位偏移、缩放、爆炸量都预编排好，切层 = 相机补间飞过去（缓动见[[补间动画与缓动函数]]）。

Q6 拖鸭腿跟手：浏览器端逆运动学（IK）是什么？

FK 正运动学（已学，[[前向运动学FK与四元数链]]）：给关节角度 → 算脚在哪。**IK 逆运动学**反过来：你直接拖脚尖到目标点 → 反算每个关节该转多少度。**官网用的是最经典的 CCD（循环坐标下降）迭代**：从脚尖往回逐关节“各转一点”——每个关节只解自己那一小段（绕轴的带符号夹角，公式 $\text{atan2}(\text{轴} \cdot (v_1 \times v_2), v_1 \cdot v_2)$ ），每次最多转 0.14 弧度防抽搐，循环 10 轮收敛。和步态叠加的方式很聪明：`ikOffset = 当前角 - 步态角`，松手后 `ikOffset` 衰减归零，鸭子自己继续走路。



CCD：踝→膝→髋 从末端往回逐个“各转一点”，转 10 轮就够到目标

Q7 "爆炸图"怎么实现? ——一人一个向量 + 缓动

每个零件存三样: 原位 **base**、爆炸方向向量 **vector** (按名字手写规则: 左壳往上、右壳往下、下巴朝前.....)、当前插值 **t**。滑杆拖动时 $\text{position} = \text{base} + \text{vector} \times t$, t 用 $0.08/\text{帧}$ 的指数平滑追目标——零件"嗖"地散开又"嗖"地装回。**爆炸图 = 给每个零件发一张"逃跑方向票", 然后集体插值。**

Q8 蓝图/解剖风格渲染: 自定义着色器只有 30 行

官网不走写实路线, 而是"工程蓝图"风: 全部零件用同一个 **ShaderMaterial**, GLSL 片元着色器干三件事——**rim 边缘光** (视线与法线夹角越大越亮, 勾轮廓)、**hatch 排线** ($\sin(\text{世界坐标} \times 245)$ 做出斜线纹理, 像铅笔稿)、**grain 颗粒** (像素坐标噪声)。配合 **EdgesGeometry** 描边线, 成品像活的工程图纸。**风格化渲染的核心: 颜色不是"贴"上去的, 是数学算出来的。**

Q9 React/Next.js 包 Three.js: 'use client' 边界

整个官网其实只有一个页面组件 `app/page.tsx`, 渲染 `<MicroduckBlueprint/>`。3D 部分整个塞进一个 **'use client'** 组件: Three.js 要碰 `window/canvas/requestAnimationFrame`, 只能在客户端跑; 服务端组件负责静态文案和 SEO。**经验法则: Three.js 场景整个生命周期挂在 `useEffect` 里, 卸载时 `dispose` 干净。**部署用 Docker Space (HF) 或 Cloudflare Workers (vnext 构建)。

Q10 鼠标指哪、信息卡出哪? ——raycast 拾取 + 诚实文案表

`pointermove` → **Raycaster** 拾取零件 → 按 mesh 名查 `inspectionCopy` 文案表 (XL330 → "20×34×26mm · 12位编码器 · TTL总线")。最值得抄的细节: 每张卡片带 **confidence 字段**——"官方仿真几何确认" / "功能确认·量产布局概念" / "原型机推测"。**官网诚实地告诉你哪句是事实、哪句是推测** (呼应[[证据优先质检ProofOverClaims]]和[[事实与判断分离]])。

Q11 相机怎么讲故事? ——聚焦舞台编排

官网 5 个章节 (整机→舵机→视觉→计算→电源), 每章是一个 **FocusStage** 数据对象: 目标点 `target`、机位 `cameraOffset`、`zoom`、爆炸量 `explosion`、贴纸。切章时相机位置/目标/爆炸量全部指数平滑飞过去——**"镜头语言"被压缩成 5 行数据, 改数据就是改分镜**。这是"内容数据化"的又一次胜利 (同[[确定性图布局]]、[[Mini-notation节奏语言]]一个思想)。

Q12 这套思路我能拿做什么？

- 给自家产品做交互官网
- 毕设/作品集 3D 展示页
- 机器人调试数字孪生面板
- 淘宝级"爆炸图"产品页
- 3D 硬件教学课件
- 只要你拿得到 **GLB 模型 + 一份关节/装配描述**，这五层方法论（资产→骨架→动作→交互→分镜）就能整体搬到任何机器人/机械产品上，源码就在 HF Space，MIT 级别开放参考。

二、能力清单（这个项目教会你什么）

能力	一句话
 GLB 资产管线	GLTFLoader 只取几何体建索引，几何与装配分离
 关节树装配	MJCF/kinematics.json → THREE.Group 树 + 铰链四元数
 策略步态回放	RL 轨迹帧按相位插值灌关节，真机数据上官网
 CCD 逆运动学	拖末端、逐关节迭代反解，且能与步态叠加
 爆炸图 + 图层	每零件一个方向向量 + 透明度权重表 + 聚焦舞台
 风格化着色器	30 行 GLSL：rim 光 + 排线 + 颗粒 = 蓝图风

三、动手实验：200 行复刻"迷你解剖官网"（已全部实测跑通）

目标：不碰 React 构建链，用"单 HTML + importmap"最小复刻官网四大能力：**GLB 加载装配、策略步态行走、图层聚焦、爆炸图**。成果在 `robot-web-anatomy\exercise\mini-duck-site\`。

第 1 步 · 备料 —— 从 repo 复制官方资产

```
cp repo/public/robot/mjlab/microduck.glb exercise/mini-duck-site/  
cp repo/public/robot/mjlab/kinematics.json exercise/mini-duck-site/
```

目的：拿到"零件库 + 装配图"。✅ 验证：两个文件在，GLB 1.28MB、json 28KB。

第 2 步 · 提取步态动作库 —— 用 node 脚本从上游 TS 库抠出 32 帧数据存成 `gait-data.js`

```
node -e "...正则抽取 MICRODUCK_WALK_FRAMES / JOINT_ORDER / CYCLE_SECONDS..."
```

✅ 验证输出：**frames: 32 • channels/frame: 18 • joints: 14 • cycle_s: 0.4061**

第 3 步 · 写单页 `index.html` —— importmap 引 three@0.185，四段核心代码：

① GLTFLoader 只取几何体 → `geometryByName` 索引；② kinematics.json → Group 树 + JointRig；③ 步态插值灌关节（**关键一步：MJCF 是 z-up，要 `root.rotation.x = -π/2` 转成 three 的 y-up**）；④ 图层按钮 + 爆炸滑杆（`position = base + vectorxt`，t 指数平滑）。

第 4 步 · 起本地服务器（ESM + fetch 不许 file:// 直开）

```
cd exercise/mini-duck-site  
python -m http.server 8617
```

✅ 验证：`curl` 四个资源全部 **200**（index/glb/kin/gait）。

第 5 步 · 无头浏览器截图自证（证据优先！）

```
msedge --headless --use-gl=swiftshader --window-size=1280,800 \  
--virtual-time-budget=25000 --screenshot=verify4.png \  
"http://localhost:8617/index.html"
```

✅ **预期结果（实测）：**小鸭直立居中行走，HUD 遥测显示「步态相位 81% · 左髋俯仰 -21.0° · 躯干滚转 -2.9° · 循环 0.812s」，状态栏「就绪 · 70 零件 · 14 关节 · 32 步态帧」。

第 6 步 · 交互验证 —— URL 参数直驱图层/爆炸

```
... --screenshot=verify5.png "http://localhost:8617/index.html?layer=MOTORS&explode=0.65"
```

✅ 预期结果 (实测) : 15 个粉色 XL330 舵机方块沿各自方向炸开, 外壳压暗成幽灵剪影; ? layer=SENSORS 则头部青色镜头组高亮。三张截图 verify4/5/6 存在 exercise 目录。

⚠ 实验踩坑记录 (两处都是真金白银的教训) : ① gait-data.js 最初用 CommonJS 的 module.exports , 浏览器 ESM 报 "does not provide an export named"——改成 export {} ; ② 第一次截图鸭子"躺倒且巨大"——MJCF 是 z-up、three 是 y-up, 必须 root 先转 -90°, 躯干步态偏移要写在旋转前的坐标系里 (y+侧偏、z+高度)。坐标系转换永远是 3D 第一坑。

四、给"以后做机器人产品官网"的落地清单

层	要准备什么	对应源码位置 (microduck-blueprint.tsx)
资产	GLB 模型 (官方 CAD/仿真导出均可)	GLTFLoader + geometryByName (L1049)
骨架	关节/装配描述 (MJCF、URDF、自写 json 都行)	KinematicBody 树 + JointRig (L1072)
动作	关键帧/策略轨迹/程序动画任选其一	MICRODUCK_WALK_FRAMES + samplePolicyGait
交互	OrbitControls 必选; IK 拖拽进阶	CCD 求解器 solveActiveIK (L1230)
分镜	FocusStage 数据表 (目标/机位/爆炸量)	LAYERS 常量 (L163)
文案	零件信息表 + confidence 诚实标注	inspectionCopy (L332)

收尾三连: ① 官网方法论 = 资产(Geometry) × 骨架(Rig) × 动作(Gait) × 交互(IK) × 分镜(Stage), 五层全部数据化; ② 上游源码在 HF Space 公开且持续更新, 克隆即学: `git clone https://huggingface.co/spaces/mishig/microduck-anatomy`; ③ 本仓库实验 `F:\reminder\robot-web-anatomy\exercise\mini-duck-site\`, ``python -m http.server`` 起来即可复现。

robot-web-anatomy · 拾遗队列 cb09e4bb · 2026-10-03 · 与 microduck (机器人本体)、microduck-hardware-guide (硬件拆解) 互为姊妹篇